

Team Name: Books.gov

Sean Gilday
Gavin McIlvaine
Elan Cote
Nathaniel Loomis
Jay Stebbins

March 19, 2025

1 Introduction

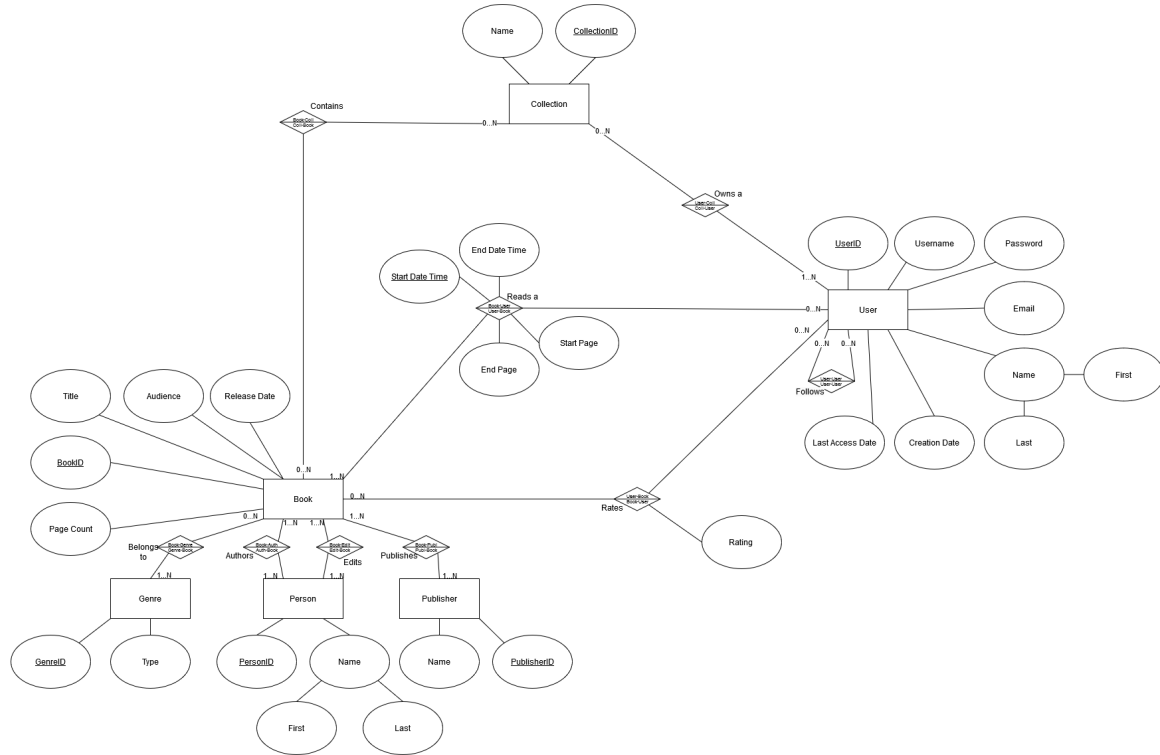
The database we are designing is meant to manage a user's reading habits and interactions. This database stores essential information about users, books, and interactions between the two. The system stores essential user information such as username, a securely stored password, first and last name, email, account creation date, and last access date. This information allows multiple users to use the system effectively. The database also manages book data, including title, intended audience (Kids, Teens, or Adults), release date, length in pages, genres, authors, editors, and publishers. In addition, the database will store unique identifiers and names of authors, editors, and publishers to ensure accurate record-keeping.

Users can take several actions within the system that get stored. When a user reads a book, the system stores the sessionID, start and end pages, date, and start and end time. Users can also rate books from 1-5, which will subsequently be stored and used to calculate an average rating. Finally, users can create and manage collections of books as well as follow and be followed by other users.

Our team will develop this database in Java and SQL and use Discord to communicate with each other.

2 Design

2.1 Conceptual Model



During the development of our EER diagram, we had many different entities and relationships to consider. Initially, we identified a rating and reading session as separate entities created by the user. However, after further consideration, we realized that those two describe the relationships between a user and a book. A user reads a book and records the start and end date and page, and a user rates a book, storing the rating. At the beginning of the development of this diagram, we had genre as an attribute of a book, but due to the wide domain of a genre, we opted to put it as its own entity with an id and the genre type as an attribute. This will allow a large domain of genres to be input into the database and allows easy access to the many different types. To consolidate our entities, we combined the editor and author entities into a single-person entity with two different relationships with a book. This streamlines the author and editor data into a single location to store person data and makes it easy to access both relations at once.

2.2 Reduction to Tables

User, Book, Collection, Author, Editor, Publisher, and Genre are all entity types that warrant their own entry in the relational model. For every many-to-many relationship in the EER diagram, an entry in the relational model is needed to link the entities that contain their respective primary keys. The entries relating to this are: User_Follows, User_Collection, Collection_Book, Book_Author, Book_Editor, Book_Publisher, Book_Genre, User_Book_Rating, User_Book_Session.

User (User_ID, Username, Password, First_Name, Last_Name, Email, Creation_Date, Last_Access_Date)

User_Follows (User_Follower, User_Followed)

User_Collection (User_ID, Collection_ID)

Book (Book_ID, Title, Audience, Length, Release_Date)

Collection (Collection_ID, User_ID, Name)

Collection_Book (Collection_ID, Book_ID)

Author (Person_ID, First_Name, Last_Name)

Book_Author (Book_ID, Person_ID)

Editor (Person_ID, First_Name, Last_Name)
 Book_Editor (Book_ID, Person_ID)
 Publisher (Publisher_ID, Name)
 Book_Publisher (Book_ID, Publisher_ID)
 Genre (Genre_ID, Genre_Type)
 Book_Genre (Book_ID, Genre_ID)
 User_Book_Rating (User_ID, Book_ID, Rating)
 User_Book_Session (User_ID, Book_ID, Start_Time, End_Time, Start_Page, End_Page)

2.3 Data Requirements/Constraints

The project database will store essential information to support the application requirements. It will manage multiple users by recording details such as username, password (secured in the final phase), first and last name, email, creation date, and last access date. For books, the database will capture the title, audience category (Kids, Teens, Adults), release date, length in pages, genres, authors, editors, and publishers. In addition, authors, editors, and publishers will be uniquely identifiable by their names. The system will also track user actions, including reading sessions (pages or locations read, start and end times, and date), star ratings for books (1-5, fixed per user per book), creation of named book collections, and user interactions like following and user following.

User:

User_ID is a primary key
 User_ID, Username, Password, First_Name, Last_Name, Email, Creation_Date, Last_Access_Date must be NOT NULL
 UserID, Username must be unique
 Last_Access_Date must not be before Creation_Date
 Users do not need to follow anyone, be followed by anyone, own any collections, rate any books, nor have any reading sessions
 User_ID is a number
 Username, Password, First_Name, Last_Name, and Email are strings
 Creation_Date and Last_Access_Date are dates

Book:

Book_ID is a primary key
 Book_ID, Title, Audience, Length, Release_Date must be NOT NULL
 Book_ID must be unique
 Books must have an author, editor, publisher, and genre
 Books do not need to be in a collection, have a rating, or be in any reading sessions.
 Audience must be either Kids, Teens, or Adults.
 Book_ID, Length are numbers
 Title, Audience are strings
 Release_Date is a date

Collection:

Collection_ID is a primary key
 User_ID is a foreign key from the User table
 Collection_ID, User_ID, Name must be NOT NULL
 Collection_ID, User_ID must be unique
 Collection_ID and User_ID are numbers
 Name is a string

Author: Person_ID is a primary key

Person_ID, First_Name, Last_Name must be NOT NULL
 Person_ID must be unique
 Person_ID is a number

First_Name and Last_Name are strings

Editor:

Person_ID is a primary key

Person_ID, First_Name, Last_Name must be NOT NULL

Person_ID must be unique

Person_ID is a number

First_Name and Last_Name are strings

Publisher:

Publisher_ID is a primary key

Publisher_ID, Name must be NOT NULL

Publisher_ID must be unique

Publisher_ID is a number

Name is a strings

Genre:

Genre_ID is a primary key

Genre_ID, Genre_Type must be NOT NULL

Genre_ID, Genre_Type must be unique

Genre_ID is a number Genre_Type is a string

User_Follows:

The aggregate of User_Follower, User_Followed is the primary key

User_Follower, User_Followed are foreign keys from the User table

User_Follower, User_Followed must be NOT NULL and be numbers

User_Collection:

The aggregate of User_ID, Collection_ID is the primary key

User_ID is a foreign key from the User table

Collection_ID is a foreign key from the Collection table

User_ID, Collection_ID must be NOT NULL and be numbers

Collection_Book:

The aggregate of Collection_ID, Book_ID is the primary key

Collection_ID is a foreign key from the Collection table

Book_ID is a foreign key from the Book table

Collection_ID, Book_ID must be NOT NULL and be numbers

Book_Author:

The aggregate of Book_ID, Person_ID is the primary key

Book_ID is a foreign key from the Book table

Person_ID is a foreign key from the Person table

Book_ID, Person_ID must be NOT NULL and be numbers

Book_Editor:

The aggregate of Book_ID, Person_ID is the primary key

Book_ID is a foreign key from the Book table

Person_ID is a foreign key from the Person table

Book_ID, Person_ID must be NOT NULL and be numbers

Book_Publisher:

The aggregate of Book_ID, Publisher_ID is the primary key

Book_ID is a foreign key from the Book table

Publisher_ID is a foreign key from the Publisher table

Book_ID, Publisher_ID must be NOT NULL and be numbers

Book_Genre:

The aggregate of Book_ID, Genre_ID is the primary key

Book_ID is a foreign key from the Book table

Genre_ID is a foreign key from the Genre table

Book_ID, Genre_ID must be NOT NULL and be numbers

User_Book_Rating:

The aggregate of User_ID, Book_ID is the primary key

User_ID is a foreign key from the User table

Book_ID is a foreign key from the Book table

User_ID, Book_ID, Rating must be NOT NULL and be numbers

Rating must be between 1 and 5 (inclusive)

User_Book_Session:

The aggregate of Start_Time, Book_ID, and User_ID is the primary Key

Book_ID and User_ID, Book are foreign keys from tables Book and User

Start_Time, User_ID, Book_ID, End_Time, Start_Page, End_Page must be NOT NULL

Start_Time, End_Time must be valid date formats

End_Time must not be before Start_Time

Start_Page must be non-negative and End_Page must not be less than Start_Page

Book_ID, User_ID, Start_Page, and End_Page are numbers

Start_Time and End_Time are dates

2.4 Sample instance data

User_ID	Username	Password	First_Name	Last_Name	Email	Creation Date	Last Access Date
1	jdoe123	ma0im43#	John	Doe	johndoe@gmail.com	2023-1-10	2024-1-15
2	JaneSmith22	g03MRQ@#	Jane	Smith	janesmith@gmail.com	2022-5-20	2024-1-18
3	AliceTheReader	03rnwpe#	Alice	Jones	alicejones@gmail.com	2021-7-15	2024-1-10
4	BobbiBobbi	U(.3rh3a	Bob	Brown	bobbrown@gmail.com	2023-12-22	2023-12-20
5	Charlie03	#10\$Nknew	Charlie	Wilson	charliewilson@gmail.com	2019-11-22	2024-1-28

Table 1: User Data Table

Book_ID	Title	Audience	Length	Release_Date
1	The Great Adventure	Adults	320	2021-05-10
2	Science Wonders	Teens	250	2022-08-14
3	Fantasy Kingdom	Kids	180	2023-01-30
4	History of Time	Adults	400	2018-11-30
5	Mystery of the Lost	Teens	290	2020-04-15

Table 2: Book Data Table

Collection_ID	User_ID	Name
1	1	John's Favorite Reads
2	2	Jane's Science Picks
3	3	Alice's Fantasy Collection
4	4	Bob's History Books
5	5	Bob's Mystery Reads

Table 3: Collection Data Table

Person_ID	First_Name	Last_Name
1	Mark	Twain
2	Isaac	Newton
3	J.K.	Rowling
4	Stephen	Hawking
5	Agatha	Christie

Table 4: Author Data Table

Person_ID	First_Name	Last_Name
1	Emily	White
2	Michael	Black
3	Sarah	Green
4	David	Blue
5	Laura	Brown

Table 5: Editor Data Table

Publisher_ID	Name
1	Penguin Books
2	HarperCollins
3	Oxford Press
4	American Library Collection
5	Random House

Table 6: Publisher Data Table

Genre_ID	Genre_Type
1	Adventure
2	Science
3	Fantasy
4	History
5	Mystery

Table 7: Genre Data Table

User_Follower	User_Followed
1	2
1	3
2	3
3	2
4	3

Table 8: User Follows Table

Collection_ID	Book_ID
1	1
2	2
3	3
4	4
5	5

Table 9: Collection Book Table

Book_ID	Person_ID
1	1
2	2
3	3
4	4
5	5

Table 10: Book Author Table

Book_ID	Person_ID
1	1
2	2
3	3
4	4
5	5

Table 11: Book Editor Table

Book_ID	Publisher_ID
1	1
2	2
3	3
4	4
5	5

Table 12: Book Publisher Table

Book_ID	Genre_Type
1	1
2	2
3	3
4	4
5	5

Table 13: Book Genre Table

User_ID	Book_ID	Rating
1	1	5
2	2	4
3	3	4
4	4	3
5	5	2

Table 14: User Book Rating Table

Start_Time	User_ID	Book_ID	End_Time	Start_Page	End_Page
2024-01-01 10:00:00	1	1	2024-01-01 11:00:00	1	50
2024-01-02 14:00:00	2	2	2024-01-02 15:00:00	1	2
2024-01-03 16:00:00	3	3	2024-01-03 17:00:00	1	31
2024-01-04 09:00:00	4	4	2024-01-04 10:00:00	1	27
2024-01-05 13:00:00	5	5	2024-01-05 14:00:00	1	23

Table 15: User Book Session Table

3 Implementation

Use this section to describe the overall implementation of your database. Include samples of SQL statements to create the tables (DDL statements) and a description of the ETL process, including examples of the SQL insert statements used to populate each table initially. Include also sample of the SQL insert statements used in your application program to insert new data in the database. Finally, add an appendix of all the SQL statements created in your application during Phase 4 and a description of the indexes created to boost the performance of your application.

For the overall implementation of our database, we used the relational model and the constraints listed above to help make our tables. We used the SERIAL type for IDs as these are automatically increasing and unique per entry.

Below is an example of how we created the users table.

```
CREATE TABLE "users" (  
    User_ID SERIAL NOT NULL PRIMARY KEY,  
    Username VARCHAR(255) NOT NULL UNIQUE,  
    Password VARCHAR(255) NOT NULL,  
    First_Name VARCHAR(255) NOT NULL,  
    Last_Name VARCHAR(255) NOT NULL,  
    Email VARCHAR(255) NOT NULL,  
    Creation_Date TIMESTAMP NOT NULL,  
    Last_Access_Date TIMESTAMP NOT NULL,  
    CHECK (Last_Access_Date >= Creation_Date)  
);
```

When creating a new user, the SQL insert statement used in our application looked like the following, where the ?'s are each value being inserted. The Creation_Date and Last_Access_Date were generated when the function were run, but the other 5 fields were obtained from the person running the application.

```
"INSERT INTO users (Username, Password, First_Name, Last_Name, Email, Creation_Date, Last_Access_Date)  
VALUES (?, ?, ?, ?, ?, ?, ?)"
```

Below is an example of how we created the collection table.

```
CREATE TABLE "collection" (  
    Collection_ID SERIAL NOT NULL PRIMARY KEY,  
    Name VARCHAR(255) NOT NULL  
);
```

When creating a new collection, the SQL insert statement used in our application looked like the following, where the ?'s are each value being inserted. There is only one ? as there is a separate table where the users table and the collection table are both referenced via foreign keys.

```
"INSERT INTO collection (Name) VALUES (?)"
```

When loading initial data into the database, the team used Mockaroo. The team generated 1000 rows of data at a time and ran the SQL insert statements in DataGrip, which would filter out invalid entries. After we generated data for all of the primary tables, we generated data for the M:N by using data generated from these tables and taking advantage of the RANDOM() function. Below are examples for 5 different tables.

Users:

```
INSERT INTO "users" (Username, Password, First_Name, Last_Name, Email, Creation_Date, Last_Access_Date)  
VALUES ('jdoe', 'hashed_password_1', 'John', 'Doe', 'jdoe@email.com', '2024-01-01 12:00:00', '2024-03-01 15:30:00')
```


Book: INSERT INTO "book" (Title, Audience, Length, Release_Date) VALUES ('The Great Adventure', 'Teens', 350, '2023-05-15 00:00:00')

Author: INSERT INTO "author" (First_Name, Last_Name) VALUES ('Mark', 'Twain')

Publisher: INSERT INTO "publisher" (Name) VALUES ('Penguin Random House')

Genre: INSERT INTO "genre" (Genre_Type) VALUES ('Fiction')

4 Data Analysis

4.1 Hypothesis

Use this section to state the objectives of your data analysis; what are the observations you are expecting to find. Note that your final observations may end up differing from your proposal, that is also a valid result.

4.2 Data Pre-processing

Use this section to describe the pre-processing steps you have performed to prepare the data for the analytics. Pre-processing steps may include: data cleaning (e.g., filling missing values, fixing outliers), formatting the data (e.g., resolving issues like inconsistent abbreviations, multiples date format in the data), combining or splitting fields, add new information (data enrichment). Explain how the data was extracted from the database for the analysis; if you used complex queries or views, or both.

4.3 Data Analytics & Visualization

Use this section to explain the process/techniques used to analyze the data, use data visualization to present the results, and explain them.

4.4 Conclusions

Use this section to explain the conclusions drawn from your data analysis.

5 Lessons Learned

Use this section to describe the issues you faced during the project and how you overcame them. Also, describe what you learned during this effort; this section, like the others, plays a critical component in determining your final grade. **The next subsection is meant to provide you with some help in dealing with figures, tables and references, as these are sometimes hard for folks new to LATEX. Your figures and tables may be distributed all over your paper (not just here), as appropriate for your paper. Please delete the following subsection before you make any submissions!**

5.1 Tables, Figures, and Citations/References

Tables, figures, and references in technical documents need to be presented correctly. As many students are not familiar with using these objects, here is a quick guide extracted from the ACM style guide. First, note that figures in the report must be original, that is, created by the student: please do not cut-and-paste figures from any other paper or report you have read or website. Second, if you do need to include figures, they should be handled as demonstrated here. State that Figure 1 is a simple illustration used in the ACM Style sample document. Never refer to the figure below (or above) because figures may be placed by LATEX at any appropriate location that can change when you recompile your source .tex file. Incidentally, in proper technical writing (for reasons beyond the scope of this

discussion), table captions are above the table and figure captions are below the figure. So the truly junk information about flavors is shown in Table 1.

6 Resources

Include in this section the resources you have used in your project beyond the normal code development such as data sets or data analytic tools (i.e. Weka, R).